Name: Scott Bosdash

Date: 10-2-2026

Assignment: CS470 Final Reflection

YouTube Presentation Link: https://youtu.be/-Ln5f9PDiKE

**Experiences and Strengths:**

This course started off well, everything was fine, getting good scores and all was well, then it turned into an absolute nightmare, and everything went wrong with life, work, and school, It was a good experience as far as learning new things could be concerned though. The steps of containerization and making a serverless web site were indeed interesting to me. I never had any interest in the cloud, nor any plan on ever using it, however, it is helpful to acquire some knowledge about it. It is a useful skillset to have in today's industry. Working with cloud services was quite frustrating to me at first, overwhelming even. It took me way too long to realize my mistakes. When I started this degree, I had hopes of developing games and potentially software, never gave too much consideration to what all that would entail.

Deploying the cloud-based services was nothing at all like what I thought it would be, I started off with a blank slate, got it to connect the front end and back end so they could communicate, and then deployed the API. The deployment was where I broke everything, every time. I kept at it for weeks, trying all that I could think of all for the same result, a non-functioning cloud-based application that only worked locally. Could not for the life of me figure out why it wouldn't work. Then it finally clicked, I updated Angular files and Node.js files, and all other components in the folder I was meant to leave as is. That led me to a path of discovery that made things much clearer. All the technical skills I learned would have come in handy some day if I were to make an app the needed to be deployed to the cloud. The lambda functions and API gateway, the S3 buckets, Docker and Docker Compose, all of those were new applications I had never even heard of, so learning how they function was kind of fun. Creating a RESTful API and using persistent data, also interesting and fun, new and exciting. DynamoDB was like Mongo, which I was already aware of previously, but different thanks to its tables. There is an entire world left unexplored when it comes to software, I will never see it all, but I enjoy seeing new applications to work with.

The fact that serverless computing exists is amazing, I read about the near perfect durability of S3 as soon as I found out about it, just because I was curious, after using it, it made me want to know more. This course gave me some new troubleshooting skills or at least honed the ones I already had. I ran into several challenges, but I figured them out in the end, which is a part of the learning process. The routing issues and deployment configuration, using the lambdas the correct way, getting the front and back end to send and receive responses from each other were all part of the challenges I ran into, some were more difficult to diagnose than others, some issues were less obvious than others. It took a lot of time, patience, and perseverance to get

through it all. Adaptability is key when it comes to developing software and games, I learned that a long time ago, but this course was definitive when it came to that. I would like to pursue a career working in the field of software engineering if it were possible but mostly, I was focused on creating games, tired of the garbage that keeps getting pushed out.

**Planning for Growth:**

Since my creation is a serverless website application it can scale automatically, it is reliable, and it has a low overhead cost. Because of that, it is adaptable and can be easily expanded upon well into the future, if it becomes necessary to do so. The way it was created with microservices allows for a brighter future for the application as opposed to inability to be upgraded or lack of scalability. Serverless cloud computing opens new pathways for growth for the project. User authentication may be added, as an example of a new direction to take, I thought about adding it myself, but I was already way behind, so I chose not to. I had a few ideas that I removed, one version of the project I had filters to filter the questions, better view of the website, it was made to look like a storefront. That idea got scrapped also, I didn't want to scrap all my ideas, but I just couldn't get anything to function how I wanted, so ideas became a secondary thought, the focus was to complete the project however I could.

Using serverless architecture can be quite efficient. The entire time while working on this project I wondered in real dollars how much it would cost to run it, I realized it is efficient, and I understand that cloud services cost money but how much? If a site runs through 100 thousand hits in a month, is it 100 dollars or more like 10,000? It doesn't matter to me in my current situation; it is just a curiosity. With the way that serverless computing handles infrastructure and scaling not to mention availability and storage components, I could understand the potential cost being rather high. If one were to acquire the storage space on their own and manage everything on their own, it would be exponentially more costly.

Paying for convenience is not a new idea, but the interesting thing about cloud services is that they cost less than traditional methods. Usually, convenience comes at a higher price, not the other way around. With AWS Lambdas being so responsive, the speed and amount of traffic becomes irrelevant, managing a high volume of traffic manually would be impossible. If the traffic becomes too much, you could always add some extra functions to combat that problem, making the functions replace the need for extra hands on the project and reducing the need for a dedicated server.

Adding new policies would help with scaling. That would also require some additional systems to be effective. New monitoring systems and perhaps some alerts wou9ld need to be included in the process. The API gateway should be able to handle any requests that come in and send them on their way while the lambdas do their thing and scale as needed, Dynamo can scale automatically in accordance with how active the database becomes on demand. Metrics are important for business, that is where CloudWatch steps in to assist. Not only for metrics but also for troubleshooting and for finding bottlenecks.

Something else that would need to be taken care of is error handling. That can be fully satisfied by monitoring and logging or retry mechanisms. One of the great things about lambdas is that they can have their own built in exception handling. Utilizing them could save countless hours of painstaking troubleshooting and failure of an application. Requests can be tried again without data loss by using dead letter queues in the case of an event not being processed or being processed incorrectly. With the help of all this monitoring, issues could be addressed before they become overwhelming which, in turn, would improve stability and reliability.

As I mentioned previously, I am curious about the cost of cloud computing, it would require a good understanding of the patterns of usage and how the resources are being utilized, poor resource management or utilization could be costly. If you base it on the number of requests, how long it takes to execute the requests, and how many resources are used, it can give a better idea of the actual cost. AWS has a tool that can estimate what the cost will be based on those assumptions along with a volume of traffic forecast. After a while, the usage data may be analyzed to predict future costs, again, based on traffic volume, resource usage, etcetera. A comparison to be made between containers and serverless architecture is that a container will almost always be more predictable since they are constantly in use with a finite number of resources being used. Most businesses are fully aware of their CPU and memory capacity which makes it easier to find the best option for their own purpose, if you are unaware of what you are paying for, it isn't good business sense. Serverless architecture can vary because of the fluctuations in the workload. More activity equals higher cost; unpredictable activity is not what you want. While that may be true, serverless cloud systems are cheaper, so if traffic is inconsistent, it still may be the best option overall. That is where the pay for performance (or execution I should say) comes in handy, low traffic, low cost.

There are other things to consider when it comes to expanding such as scaling. Microservices are a great way to show this as they scale independently of each other. Teamwork is not compromised when using microservices, different teams may work on different components at the same time. This becomes important for larger businesses, they don't want any down time when updating, one team can deploy an update on one component while another team works on a separate component's update. The downside, since there always is one, is that it makes the system more complex. The more services there are, the more communication that is required, the more that can go wrong. In a perfect environment, everything works well together, but it could be a nightmare. Lower costs, less management of infrastructure, and automatic scaling may be good enough reasons to face potential issues, it is very much situationally dependent. The fact that cold start latency and limitations exist, or dependency of vendors, unpredictable costs if growth becomes unstable or unusually fast, and limitations of execution exist, containers are there as another valid option. Containers are flexible like MongoDB, they do however require a little extra care like an infant. Managing containers and babysitting them is an additional cost, but it can be the right path for the right business.

One of the most important concepts of cloud computing is elasticity when it comes to future growth. Resources that are elastic will scale when they need to, whether they are scaling up during peak times or down during leaner times. In that way, it is a great cost reduction resource for pay per service models. Spending money on an infrastructure that is not beneficial is never a good idea, elastic resources alleviate that problem. No need for a huge upfront cost when scaling if it is done automatically, good idea and it saves some money, win-win scenario. The application grows up as it should while maintaining its performance, reliability, security, and the business is financially stable through it all.